

Security Audit & Remediation Report

Web Application — Production Security Hardening

Generated: 2026-08-31 10:57 UTC

Executive Summary

Critical	2
High	4
Medium	3
Low / Info	2
All Fixed	✓ YES
Django check --deploy warnings	1 (intentional — SSL handled by Cloudflare)

Phase 1: Settings & Infrastructure Hardening (Previously Fixed)

SECRET_KEY	Moved from hardcoded insecure string to .env environment variable with generated 50+ char key
DEBUG	Set to False by default; controlled via .env DEBUG= variable
ALLOWED_HOSTS	Changed from ["*"] to explicit domain list parsed from .env
DB_PASSWORD	Moved to .env; raises KeyError at startup if missing
SESSION_COOKIE_SECURE	Enabled in production (non-DEBUG) mode
CSRF_COOKIE_SECURE	Enabled in production mode
SECURE_HSTS_SECONDS	Set to 31,536,000 (1 year) with subdomains + preload
X_FRAME_OPTIONS	Changed from SAMEORIGIN to DENY
robots.txt	Added Django view blocking 15+ web crawlers and AI scrapers

X-Robots-Tag header	RobotsHeaderMiddleware injects noindex/noai on every response
Meta robots tags	Added noindex/noai to all 7 HTML templates
.env in .gitignore	Secrets file explicitly excluded from version control

Phase 2: Code Vulnerability Fixes

CRITICAL	#1. Open Redirect in sanitize_next_url() FIXED File: core/views.py:L146 The ?next= parameter accepted absolute URLs enabling phishing redirects after login. Fixed by parsing with urlparse() and rejecting any URL with a scheme or netloc. Before: Accepted any string including //evil.com or https://attacker.com After: urlparse() validates no scheme/netloc; protocol-relative URLs blocked
CRITICAL	#2. Scquizz Admin APIs Had No Authentication FIXED File: scquizz/views.py (all admin endpoints) 10 admin API endpoints (create/delete sessions, activate polls, delete messages etc.) were callable by anyone. Added _require_moderator() helper returning 401/403 before any logic runs. Before: POST/DELETE/PATCH to sessions, polls, messages required no login After: _require_moderator() guard added to every mutating endpoint
HIGH	#3. WebSocket Accepted Unauthenticated Connections FIXED File: scquizz/consumers.py:L10 Any client could connect to the WebSocket and receive real-time quiz updates including student messages, poll changes, and active user counts. Before: connect() called accept() for all incoming connections After: Unauthenticated connections closed with code 4001 before accept()
HIGH	#4. GET /api/messages Exposed All Student Messages Without Login FIXED File: scquizz/views.py:messages_view The GET endpoint for student messages had no authentication requirement. Any unauthenticated visitor could retrieve all message history. Before: GET returned all messages with no authentication check After: Returns 401 if request.user.is_authenticated is False

HIGH	#5. Open Redirect in logout_view via HTTP_REFERER File: core/views.py:logout_view HTTP_REFERER is fully attacker-controlled. A crafted page could trigger a logout and redirect to a phishing site. The logout view now only reads the ?next POST param and defaults to /login/. Before: HTTP_REFERER header used directly for redirect destination after logout After: HTTP_REFERER completely removed; only POST ?next= (sanitized) used	FIXED
HIGH	#6. CSRF Logout via GET Request File: core/views.py:logout_view GET-based logout enables CSRF logout attacks — an attacker's page embeds an image pointing to /logout/ forcing victim logout. Now only POST requests trigger actual logout. Before: GET /logout/ logged the user out (CSRF trivial via tag) After: Non-POST requests to /logout/ silently redirect to /login/ without logout	FIXED
HIGH	#7. No Rate Limit on Login Endpoint (Brute Force) File: scquizz/middleware.py The login form had no rate limiting, allowing unlimited password guesses. Fixed by adding a fixed 10 req/min/IP Redis-backed limit to /login/ and /ict/scquizz/login. Before: RateLimitMiddleware only protected /api/messages After: Login paths limited to 10 POST attempts/min/IP via Redis counter	FIXED
MEDIUM	#8. IP Spoofing in Login Audit Logs File: core/signals.py Login audit logs recorded the first value from X-Forwarded-For which any attacker can set. Now uses CF-Connecting-IP (injected by Cloudflare, can't be spoofed by clients) first, matching the rate-limit middleware behavior. Before: X-Forwarded-For header read directly — trivially spoofable After: CF-Connecting-IP checked first (Cloudflare), then X-Forwarded-For	FIXED
MEDIUM	#9. Login Logs API: Unbounded ?limit Parameter (DoS) File: core/views.py:api_login_logs_list A staff user could pass ?limit=999999 to fetch millions of log rows in one request, potentially causing OOM or extreme DB load. Capped at 500. Before: limit = int(request.GET.get('limit', 100)) # No maximum After: limit = min(int(request.GET.get('limit', 100)), 500)	FIXED

MEDIUM	#10. Internal Exception Details Exposed to Clients FIXED
	File: core/views.py, scquizz/views.py (all except blocks) All exception handlers passed str(e) directly to the client response, leaking internal details like Django ORM error messages, column names, or constraint names. Now uses generic messages with full logging. Before: return JsonResponse({'error': str(e)}, ...) — exposes DB errors, table names After: Generic Thai error message returned; full error logged via <code>logger.error(exc_info=True)</code>
LOW	#11. Database Backup File in Repository FIXED
	File: backup_sqlite_20260828_200913.json + .gitignore A plaintext database backup was present in the workspace and not excluded from version control. Committed backups leak schema and potentially sensitive data. Before: .gitignore did not exclude backup files After: Added <code>*backup*.json</code>, <code>*backup*.sql</code>, <code>*dump*.sql</code> patterns to .gitignore

Files Modified

core/settings.py	Secrets to env vars, DEBUG=False, HSTS, secure cookies, core logger
core/views.py	Fixed <code>sanitize_next_url</code> , logout POST-only, logs cap, generic errors, urllib import
core/signals.py	Fixed IP resolution (CF-Connecting-IP first), capped User-Agent length
core/middleware.py	New file: RobotsHeaderMiddleware (X-Robots-Tag on all responses)
core/urls.py	Added <code>/robots.txt</code> route
scquizz/views.py	Added <code>_require_moderator()</code> guard to all admin APIs, GET <code>/api/messages</code> auth, generic errors
scquizz/consumers.py	WebSocket now rejects unauthenticated connections (close 4001)
scquizz/middleware.py	Added login brute-force rate limiting (10/min/IP)
core/templates/core/landing.html	Added <code>noindex/noai</code>
core/templates/core/login.html	Added <code>noindex/noai</code>
core/templates/core/admin_dashboard.html	Added <code>noindex/noai</code>
core/templates/core/permission_denied.html	Added <code>noindex/noai</code>
scquizz/template/*	Added <code>noindex/noai</code> to 3 templates
.env	New file: all production secrets (gitignored)
.gitignore	Added <code>.env</code> , <code>*backup*.json</code> , <code>*backup*.sql</code> patterns

All fixes verified with **python manage.py check --deploy** — 0 errors, 1 intentional warning (SECURE_SSL_REDIRECT disabled because Cloudflare handles SSL termination).